Setting Up VS Code for PyTorch & Jupyter Notebooks

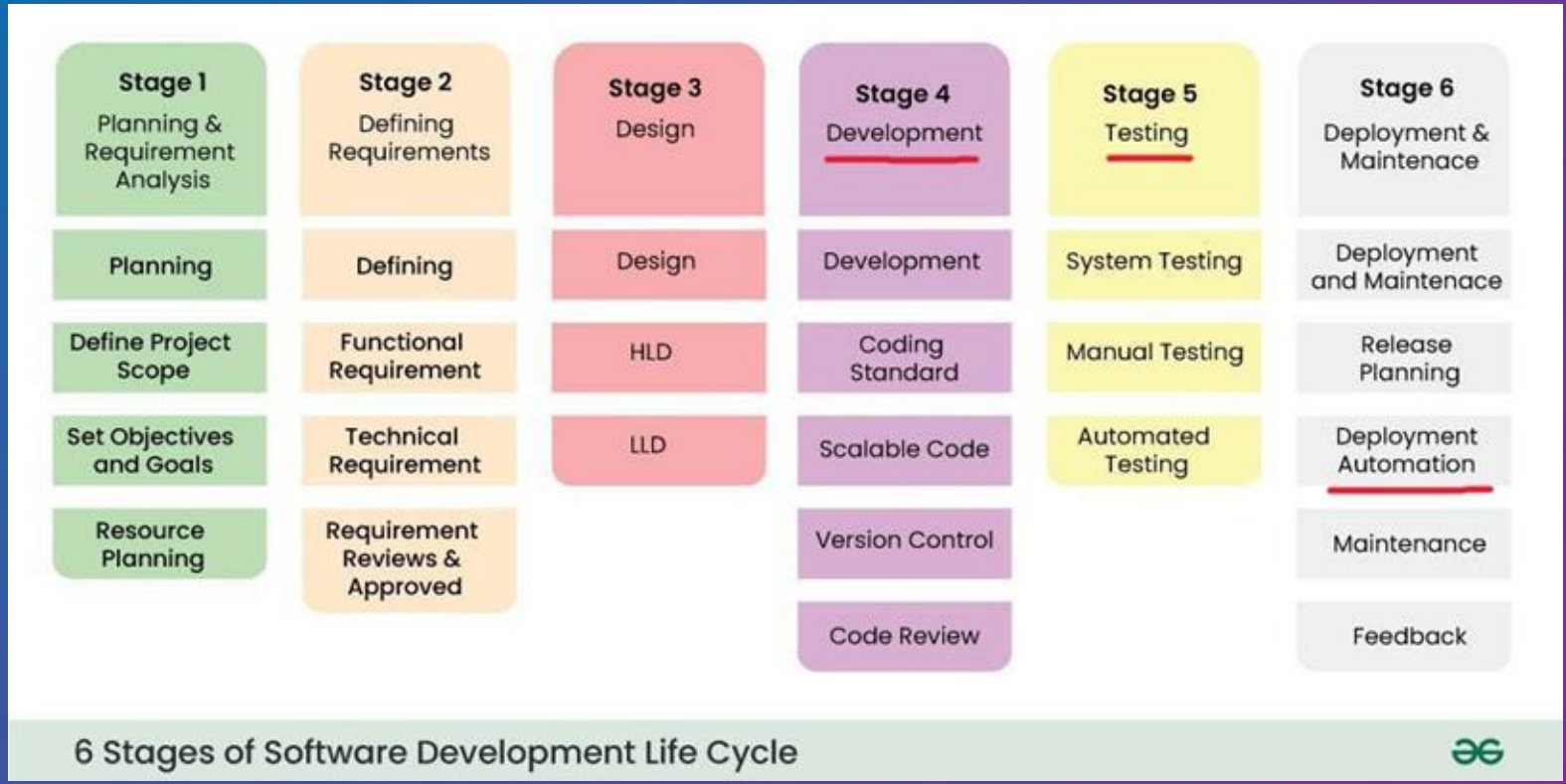
A Step-by-Step Guide to a Deep Learning Environment



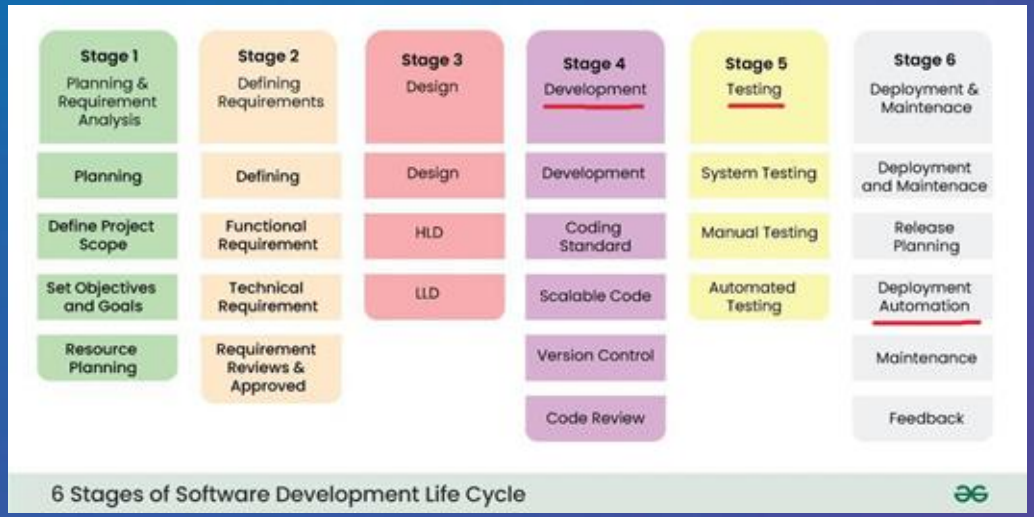




Why does IDEs Exist?

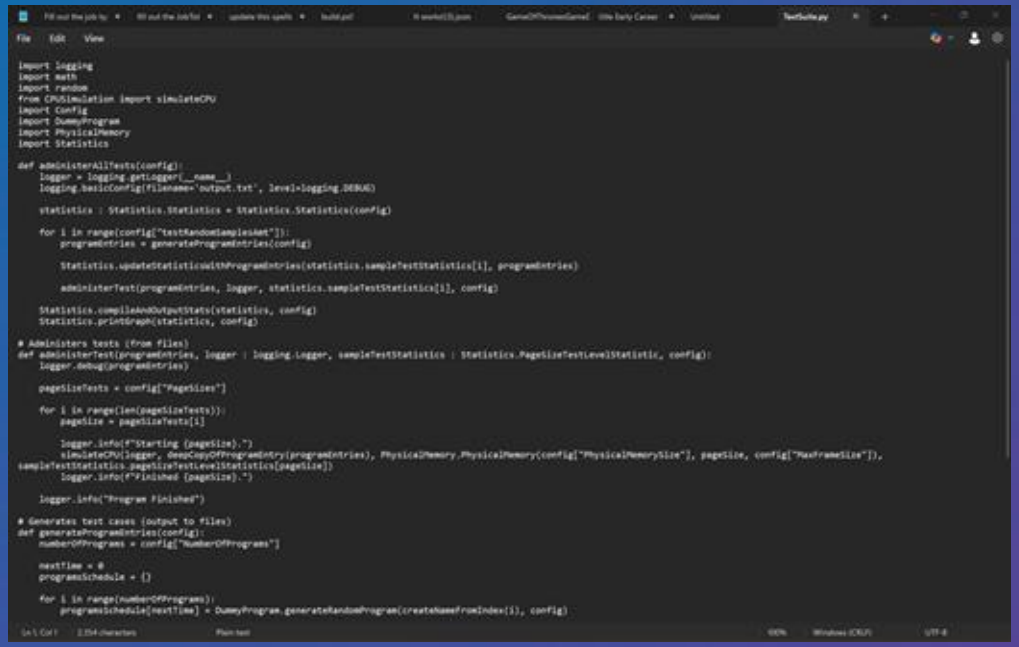


Why does IDEs Exist?



- The purpose of Integrated Development Environments or IDEs are to enable programmers to create software more efficiently
- Regardless of the your environment, exposure to the basic and advanced tools of IDEs can enable you to create a more efficient development environment for your application

IDEs: Basic to Sophisticated



```
import logging
import math
import random
from CPUSimulation import simulateCPU
import Config
import DummyProgram
import PhysicalMemory
import Statistics

def administerAllTests(config):
    logger = logging.getLogger(__name__)
    logging.basicConfig(filename='output.txt', level=logging.DEBUG)

    statistics : Statistics.Statistics = Statistics.Statistics(config)

    for i in range(config["testRandomSamplesAmt"]):
        programEntries = generateProgramEntries(config)

        Statistics.updateStatisticsWithProgramEntries(statistics.sampleTestStatistics[i], programEntries)

        administerTest(programEntries, logger, statistics.sampleTestStatistics[i], config)

    Statistics.compileAndOutputStats(statistics, config)
    Statistics.printGraph(statistics, config)

# Administers tests (from files)
def administerTest(programEntries, logger : logging.Logger, sampleTestStatistics : Statistics.PageSizeTestLevelStatistic, config):
    logger.debug(programEntries)

    pageSizeTests = config["PageSizes"]

    for i in range(len(pageSizeTests)):
        pageSize = pageSizeTests[i]

        logger.info(f"Starting {pageSize}.")
        simulateCPU(logger, deepCopyOfProgramEntry(programEntries), PhysicalMemory.PhysicalMemory(config["PhysicalMemorySize"], pageSize, config["MaxFrameSize"]),
sampleTestStatistics.pageSizeTestLevelStatistics[pageSize])
        logger.info(f"Finished {pageSize}.")

    logger.info("Program Finished")

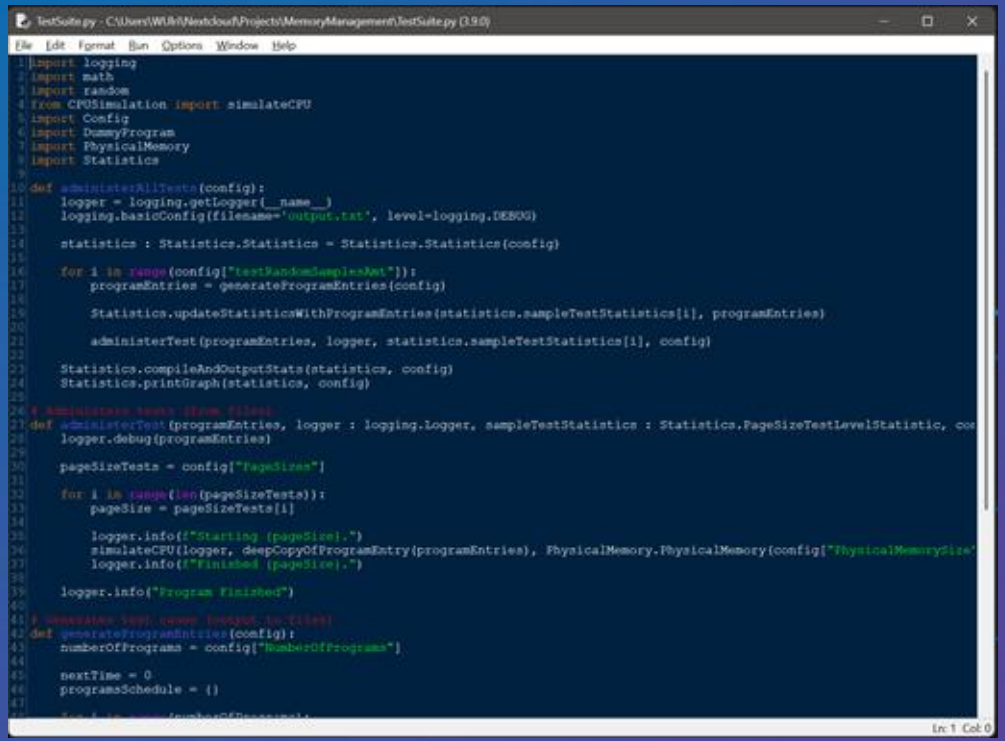
# Generates test cases (output to files)
def generateProgramEntries(config):
    numberOfPrograms = config["NumberOfPrograms"]

    nextTime = 0
    programsSchedule = {}

    for i in range(numberOfPrograms):
        programsSchedule[nextTime] = DummyProgram.generateRandomProgram(createNameFromIndex(i), config)
```

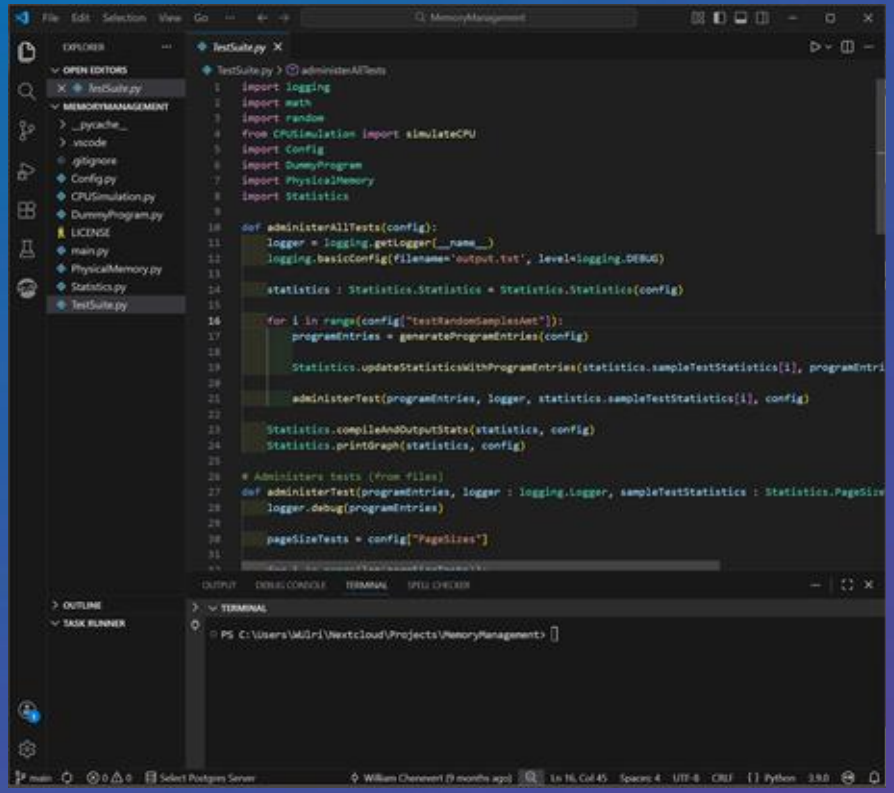
- Basic text editors and terminals already come pre-installed
- Supports large range of file types and file encodings
- Does not have features such as autocomplete, syntax highlighting, and error checking

IDEs: Basic to Sophisticated



- Python comes with a build-in text editor that supports simple script development
- Basic syntax highlighting, coding macros, code execution, and shell access
- Excels at being plug and play, simple, and approachable python execution

IDEs: Basic to Sophisticated



- Ex: VSCode, Neovim, PyCharm
- These programs provide a-one-stop-shop for the whole development cycle
- Under the hood, VSCode is a very pretty wrapper for a text editor, file preview, and terminal. Virtually any extension you use can be rewritten with a terminal command
- Large capacity for customization

VSCode Deep-Dive: Coding

- IntelliSense - A sophisticated code-completion tool that provides intelligent suggestions for variable names, functions, properties, and parameters based on the context of your code.
- Static Error Detection - Also known as linting, this feature analyzes your code as you type to highlight syntax errors, potential bugs, or style violations without needing to run the program.
- Refactoring Support - Tools that allow you to restructure existing code safely, such as renaming variables across multiple files or extracting a chunk of code into a new function.
- Code Snippets - Pre-defined templates or "shorthand" blocks of code that you can quickly insert for common patterns (like loops or class definitions) to avoid repetitive typing

VSCode Deep-Dive: Coding



Visual Studio Code

Keyboard shortcuts for Windows

General

Ctrl+Shift+P, F1	Show Command Palette
Ctrl+P	Quick Open, Go to File...
Ctrl+Shift+N	New window/instance
Ctrl+Shift+W	Close window/instance
Ctrl+,	User Settings
Ctrl+K Ctrl+S	Keyboard Shortcuts

Basic editing

Ctrl+X	Cut line (empty selection)
Ctrl+C	Copy line (empty selection)
Alt+ ↑ / ↓	Move line up/down
Shift+Alt + ↓ / ↑	Copy line up/down
Ctrl+Shift+K	Delete line
Ctrl+Enter	Insert line below
Ctrl+Shift+Enter	Insert line above
Ctrl+Shift+\	Jump to matching bracket
Ctrl+] / [	Indent/outdent line
Home / End	Go to beginning/end of line
Ctrl+Home	Go to beginning of file
Ctrl+End	Go to end of file
Ctrl+↑ / ↓	Scroll line up/down
Alt+PgUp / PgDn	Scroll page up/down
Ctrl+Shift+[	Fold (collapse) region
Ctrl+Shift+]	Unfold (uncollapse) region
Ctrl+K Ctrl+[	Fold (collapse) all subregions
Ctrl+K Ctrl+]	Unfold (uncollapse) all subregions
Ctrl+K Ctrl+0	Fold (collapse) all regions
Ctrl+K Ctrl+J	Unfold (uncollapse) all regions
Ctrl+K Ctrl+C	Add line comment
Ctrl+K Ctrl+U	Remove line comment
Ctrl+/	Toggle line comment
Shift+Alt+A	Toggle block comment
Alt+Z	Toggle word wrap

Navigation

Ctrl+T	Show all Symbols
Ctrl+G	Go to Line...
Ctrl+P	Go to File...
Ctrl+Shift+O	Go to Symbol...
Ctrl+Shift+M	Show Problems panel
F8	Go to next error or warning
Shift+F8	Go to previous error or warning
Ctrl+Shift+Tab	Navigate editor group history
Alt+ ← / →	Go back / forward
Ctrl+M	Toggle Tab moves focus

Search and replace

Ctrl+F	Find
Ctrl+H	Replace
F3 / Shift+F3	Find next/previous
Alt+Enter	Select all occurences of Find match
Ctrl+D	Add selection to next Find match
Ctrl+K Ctrl+D	Move last selection to next Find match
Alt+C / R / W	Toggle case-sensitive / regex / whole word

Multi-cursor and selection

Alt+Click	Insert cursor
Ctrl+Alt+ ↑ / ↓	Insert cursor above / below
Ctrl+U	Undo last cursor operation
Shift+Alt+I	Insert cursor at end of each line selected
Ctrl+L	Select current line
Ctrl+Shift+L	Select all occurences of current selection
Ctrl+F2	Select all occurences of current word
Shift+Alt+→	Expand selection
Shift+Alt+←	Shrink selection
Shift+Alt + (drag mouse)	Column (box) selection
Ctrl+Shift+Alt + (arrow key)	Column (box) selection
Ctrl+Shift+Alt + PgUp/PgDn	Column (box) selection page up/down

Rich languages editing

Ctrl+Space	Trigger suggestion
Ctrl+Shift+Space	Trigger parameter hints
Shift+Alt+F	Format document
Ctrl+K Ctrl+F	Format selection
F12	Go to Definition
Alt+F12	Peek Definition
Ctrl+K F12	Open Definition to the side
Ctrl+.	Quick Fix
Shift+F12	Show References
F2	Rename Symbol
Ctrl+K Ctrl+X	Trim trailing whitespace
Ctrl+K M	Change file language

Editor management

Ctrl+F4, Ctrl+W	Close editor
Ctrl+K F	Close folder
Ctrl+\	Split editor
Ctrl+ 1 / 2 / 3	Focus into 1st, 2nd or 3rd editor group
Ctrl+K Ctrl+ ←/→	Focus into previous/next editor group
Ctrl+Shift+PgUp / PgDn	Move editor left/right
Ctrl+K ← / →	Move active editor group

File management

Ctrl+N	New File
Ctrl+O	Open File...
Ctrl+S	Save
Ctrl+Shift+S	Save As...
Ctrl+K S	Save All
Ctrl+F4	Close
Ctrl+K Ctrl+W	Close All
Ctrl+Shift+T	Reopen closed editor
Ctrl+K Enter	Keep preview mode editor open
Ctrl+Tab	Open next
Ctrl+Shift+Tab	Open previous
Ctrl+K P	Copy path of active file
Ctrl+K R	Reveal active file in Explorer
Ctrl+K O	Show active file in new window/instance

Display

F11	Toggle full screen
Shift+Alt+0	Toggle editor layout (horizontal/vertical)
Ctrl+ + / -	Zoom in/out
Ctrl+B	Toggle Sidebar visibility
Ctrl+Shift+E	Show Explorer / Toggle focus
Ctrl+Shift+F	Show Search
Ctrl+Shift+G	Show Source Control
Ctrl+Shift+D	Show Debug
Ctrl+Shift+X	Show Extensions
Ctrl+Shift+H	Replace in files
Ctrl+Shift+J	Toggle Search details
Ctrl+Shift+U	Show Output panel
Ctrl+Shift+V	Open Markdown preview
Ctrl+K V	Open Markdown preview to the side
Ctrl+K Z	Zen Mode (Esc Esc to exit)

Debug

F9	Toggle breakpoint
F5	Start/Continue
Shift+F5	Stop
F11 / Shift+F11	Step into/out
F10	Step over
Ctrl+K Ctrl+I	Show hover

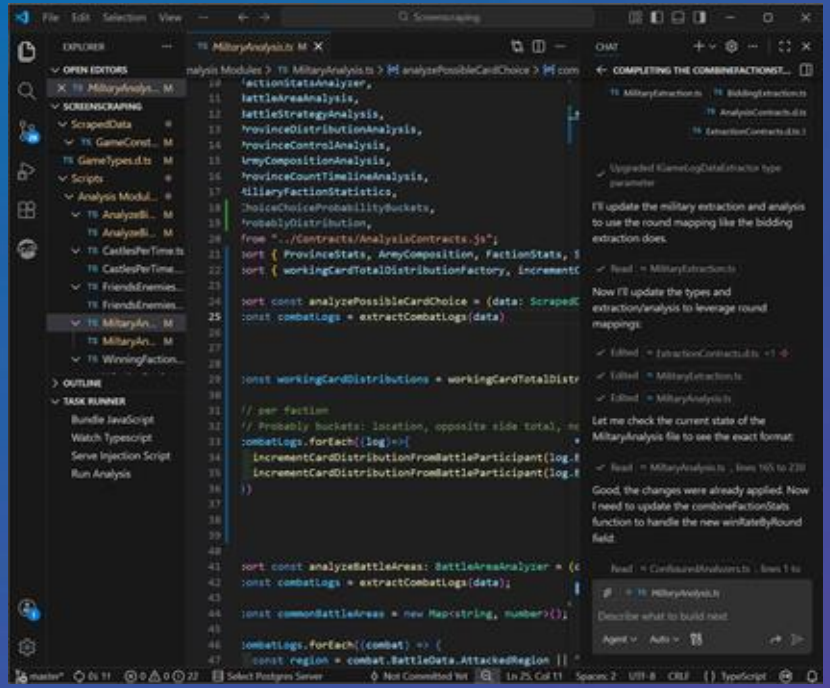
Integrated terminal

Ctrl+`	Show integrated terminal
Ctrl+Shift+`	Create new terminal
Ctrl+C	Copy selection
Ctrl+V	Paste into active terminal
Ctrl+↑ / ↓	Scroll up/down
Shift+PgUp / PgDn	Scroll page up/down
Ctrl+Home / End	Scroll to top/bottom

Other operating systems' keyboard shortcuts and additional unassigned shortcuts available at aka.ms/vscodekeybindings

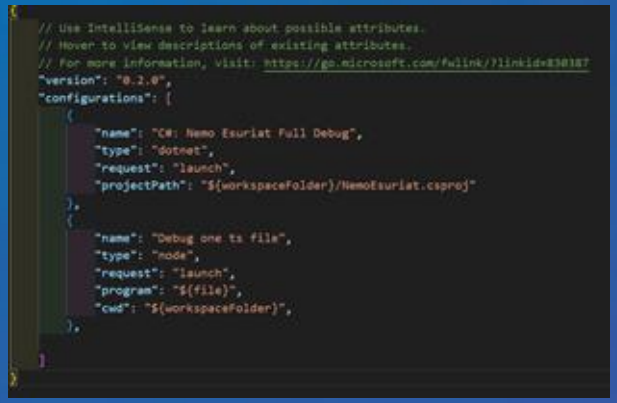
- Moving your hand from your keyboard to your mouse to navigate the UI to get to the next coding section may take 1 second - 15 minute (in large code bases), which add up
- Macros are huge time saver in this respect: Inter - Intra code file Navigation, multiselect, window manipulation, debugging hotkeys, etc

VSCode Deep-Dive: Coding



- VSCode integrates with AI Agents
- AI is great for language and library exploration, simple prototyping, test writing, data manipulation, and refactoring
- Excels in highly documented and/or small (<10k lines) code bases
- Drag and Drop files that you want to attach to prompt (saves on copy and paste, and context switching)
- Presents a file diff when finished

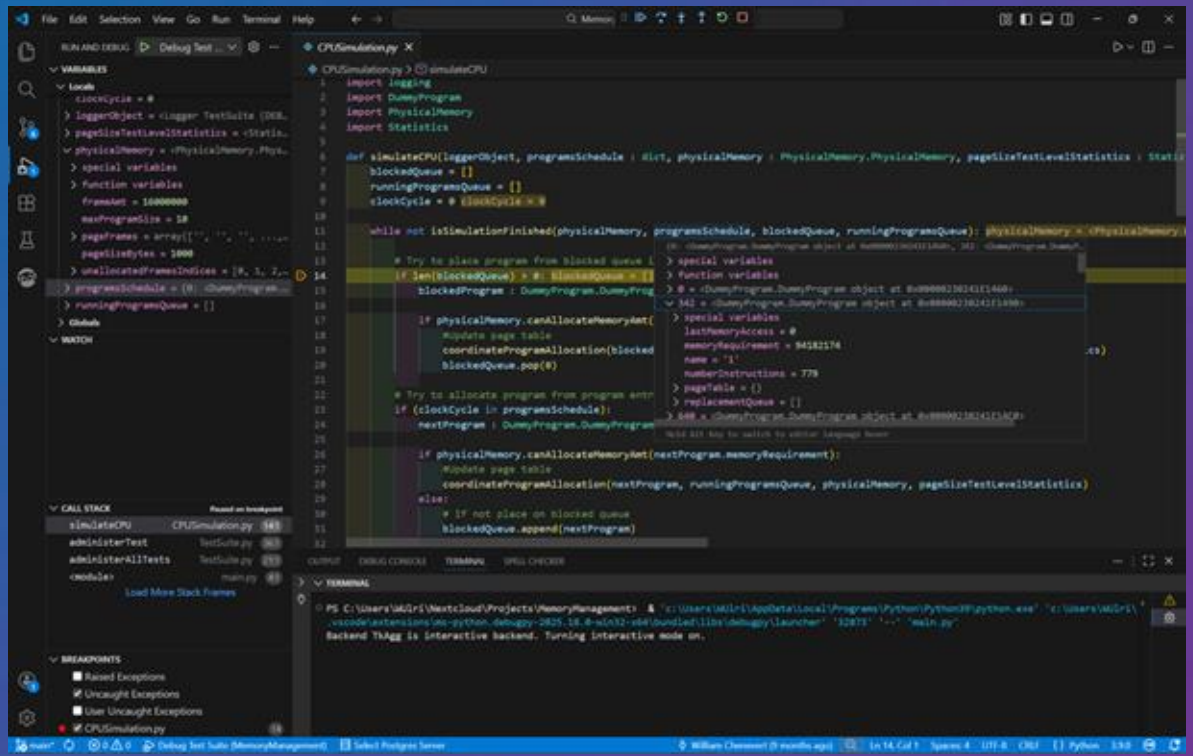
VSCode Deep-Dive: Building



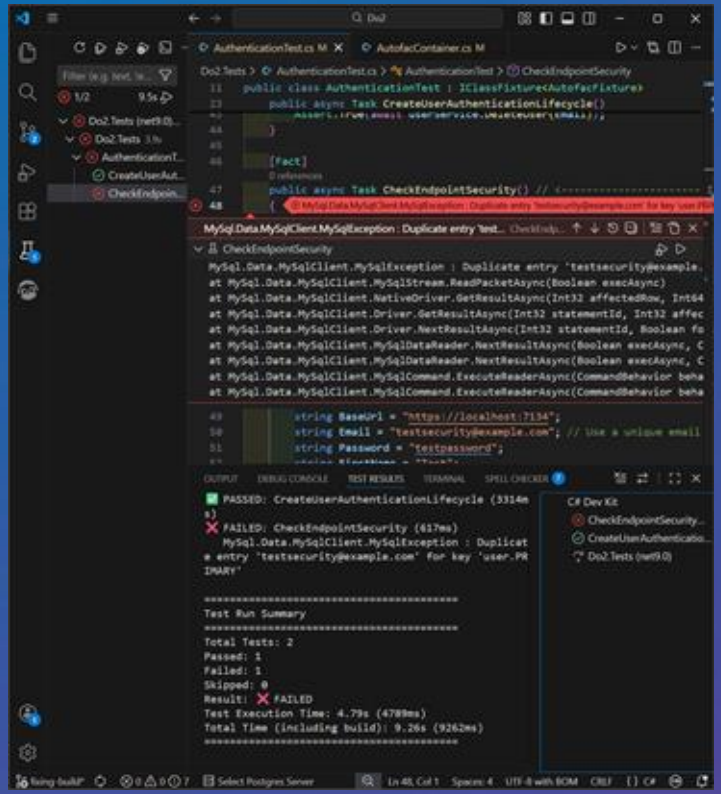


- Typing the same commands over and over again is time consuming, boring, and prone to error (I once completely deleted a 50 GB production database)
- So simplify and standardize your code execution / building like to one or two clicks
- Use launch.json and task.json

VSCode Deep-Dive: Debugging

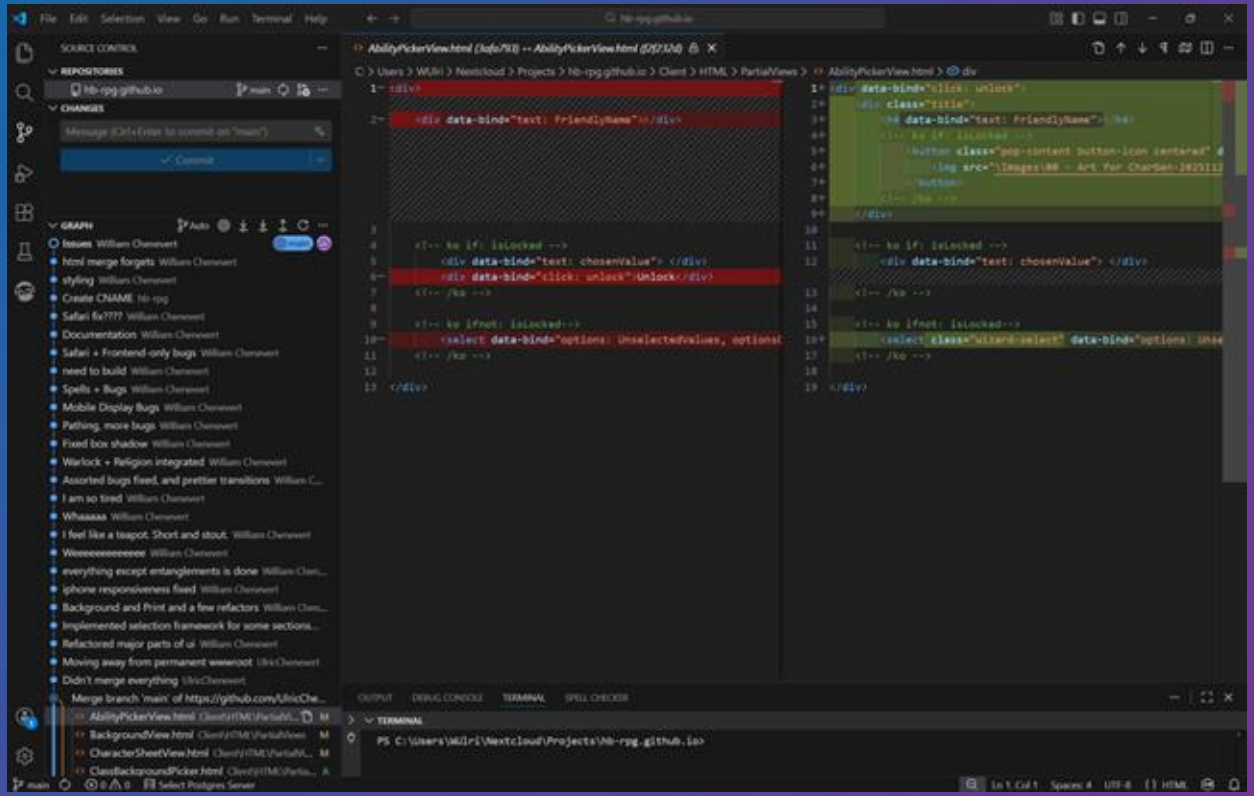


VSCode Deep-Dive: Testing



- Especially for programs with a lot of independent/interdependent features, unit tests become critical.
- VSCode integrates with a lot of languages / libraries to support test code execution
- This allows to see at a glance what code changes breaks which features and why
- Just create a test object see if your function performs as expected

VSCode Deep-Dive: Git



VSCode Deep-Dive: Extensions

- If you want some feature added to VSCode, you can program it!
- My favorite Extensions:
  - Task Runner: Automate task execution with a single click!
  - Code Spell Checker: IDK how to speel and its great static code analysis
  - Echo API: Free and powerful API Interface (Great for backend web)
  - Live Server: Agile and very lightweight static-file web server
  - Intent Rainbow: Pretty colors
  - Material Icon Theme: Pretty pictures

VSCode Deep-Dive: Example



PyTorch: An Introduction

A flexible and powerful machine learning library



- Developed by Facebook's AI Research Lab (FAIR)
- Open-source Python library for deep learning
- Key features: dynamic computation graphs, strong GPU acceleration, and a rich ecosystem

What is PyTorch

A scientific computing framework

- A replacement for NumPy to use the power of GPUs
- A deep learning research platform that provides maximum flexibility and speed





Seamlessly integrates with the Python data science stack

PyTorch vs TensorFlow

PyTorch

- Developed by Meta AI
- Dynamic computation graph (eager execution)
- More ‘Pythonic’ and flexible
- Popular in research and academia

TensorFlow

- Developed by Google
- Static computation graph (historically)
- Mature ecosystem for production (TFX, TF Lite)
- Steeper learning curve, powerful for large-scale deployment

Installation and Requirements

Installation via pip

```
pip install torch
torchvision
torchaudio
```

This installs the CPU-only version by default. For GPU support, see the official PyTorch website.

System & Software Requirements

- Python 3.7 or later
- 64-bit Operating System (Linux, macOS, Windows)
- Recommended: NVIDIA GPU with CUDA for acceleration (optional)

Basic Usage: Tensors

The fundamental data structure in PyTorch

Creating a Tensor

```
import torch

# Create a tensor from a list
x = torch.tensor([[1., 2.], [3., 4.]])
print(x)
```

Basic Operations

```
# Perform a simple operation (addition)
y = x + 5
print(y)
```

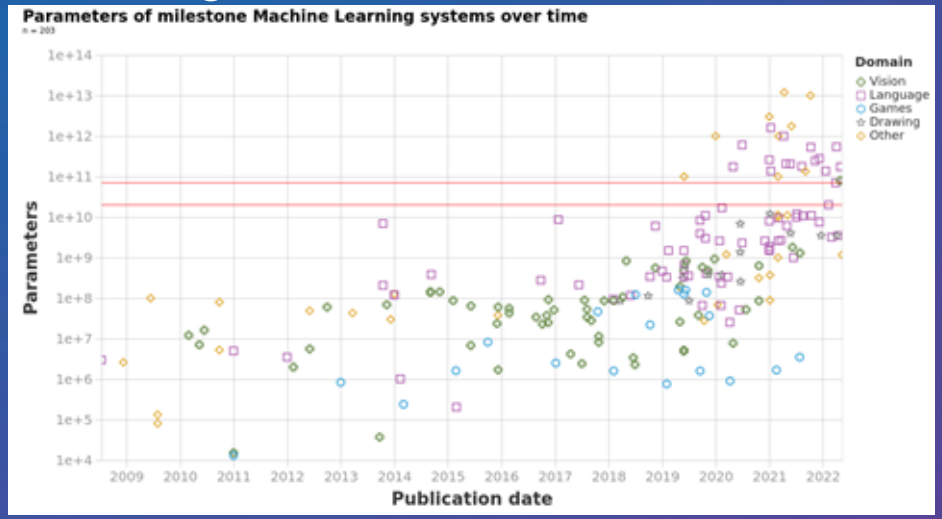
Output:

```
tensor([[1., 2.],
        [3., 4.]])
tensor([[6., 7.],
        [8., 9.]])
```

Parallelization

PyTorch can compute on the GPU, enabling massive speed improvements when compared to NumPy

Working with massive tensors

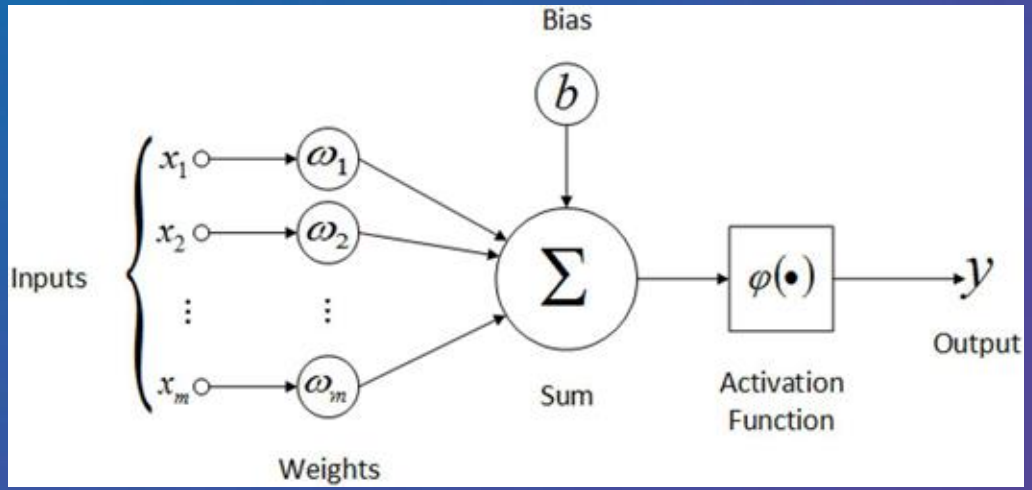


Utilizes CUDA



Forward Pass

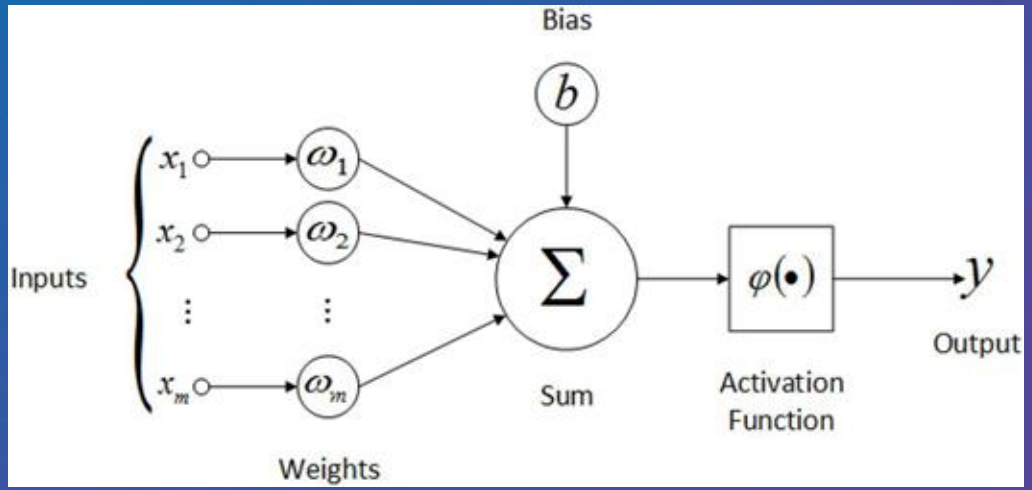
Passing input data through the network to get a guess (prediction)



1. Input to the corresponding weights
2. Bias term is added to the weighted sum to shift the activation
3. Total sum is passed through non-linear activation function
4. The final value (y) is the networks' prediction for that layer

Backpropagation

Learning by adjusting weights based on prediction errors



1. Determine the difference between prediction and actual target
2. Propagate the error back through the network layers
3. Use the error gradient to update the weights and biases
4. Repeat forward and backward to minimize error over time

Short Demo: A Simple Neural Network

```
import torch
import torch.nn as nn

# Define a simple feedforward neural network
class SimpleNet(nn.Module):
    def __init__(self):
        super(SimpleNet, self).__init__()
        self.fc1 = nn.Linear(10, 5) # Input layer to hidden layer
        self.relu = nn.ReLU()       # Hidden layer to output layer
        self.fc2 = nn.Linear(5, 2)
    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        return x

# Instantiate the model and create a sample input
model = SimpleNet()
input_tensor = torch.randn(1, 10)

# Perform a forward pass
output = model(input_tensor)
print(output)
```

Output:
tensor([[0.1234, -0.5678]], grad_fn=<AddmmBackward0>)
(Note: Output values will vary slightly due to random initialization)

Phase 1: Prerequisites

Install Python



- Download from python.org
- Important: Check ”Add Python to PATH” on Windows

Install VS Code



- Download from code.visualstudio.com

Phase 2: Setup VS Code Extensions

Step 1: Open VS Code



Step 2: Open Extensions View



Click Extensions icon on left sidebar (or press Ctrl+Shift+X)

Step 3: Search and Install



- Search for "Python" and "Jupyter"
- Install both extensions by Microsoft

Phase 3: Create a Project & Virtual Environment

Step 1: Open a Folder



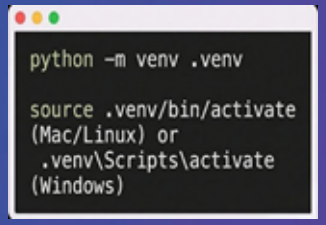
Go to File > Open
Select an empty folder

Step 2: Open the Terminal



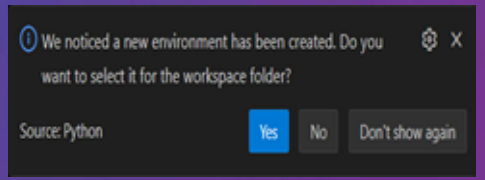
Press Ctrl + ` (backtick) or Terminal

Step 3: Run Commands



Create environment:
python -m venv .venv
Activate: source .venv/bin/activate or .venv\Sripts\activate

Step 4: Create the venv



VS Code pop-up:
“We noticed a new environment…”
Click “Yes”

Phase 4: Installing PyTorch

Step 1:
Go to the generator

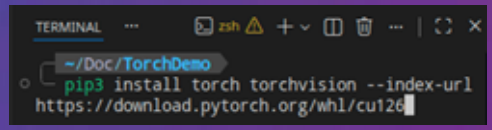
Visit the official PyTorch get started page

Step 2:
Select your settings



Select: Stable, OS, Pip, Python, Compute Platform

Step 3:
Run in VS Code

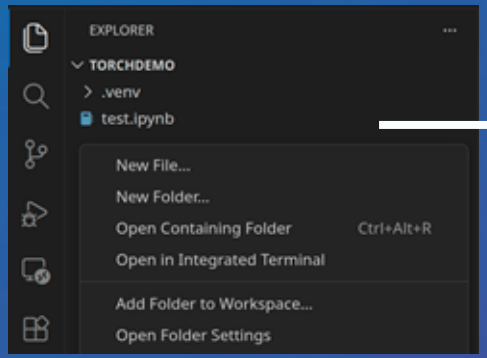


Copy the ‘Run this Command’
Paste into VS Code terminal and press ente

Note: If using Jupyter, also run ‘pip install ipykernel’

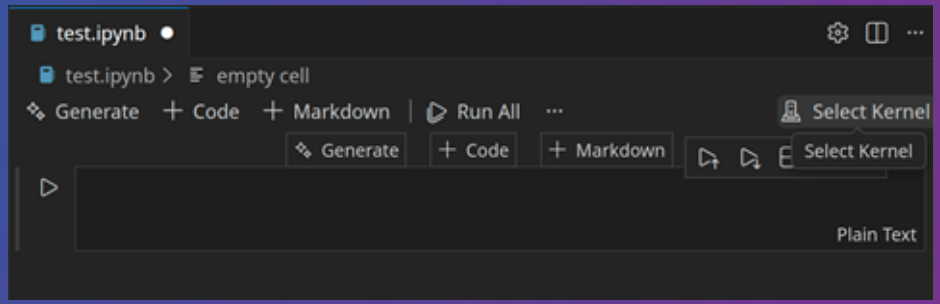
Phase 5: Creating and Configuring the Notebook

Step 1:
Create the file



In File Explorer, right-click > New File
Name it ‘test.ipynb’

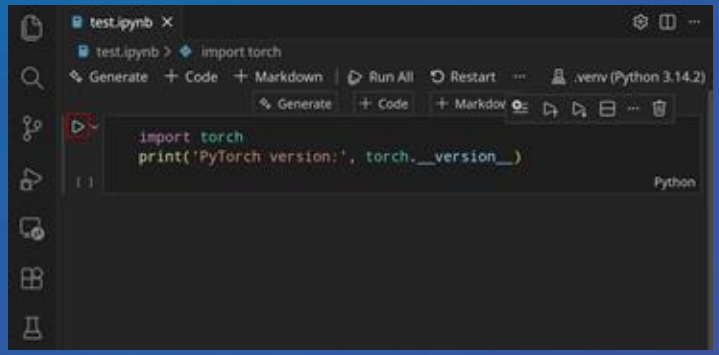
Step 2:
Select the Kernel



Open test.ipynb
Click ‘Select Kernel’ (top right)
Choose the .venv environment

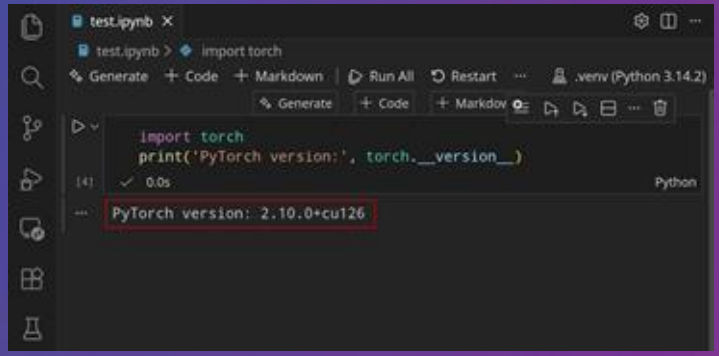
Phase 6: Writing and Running PyTorch Code

Step 1:
Paste the code



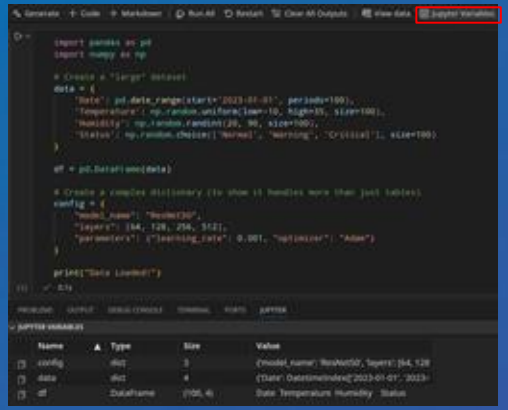
Click inside the first code cell
Write the verification code

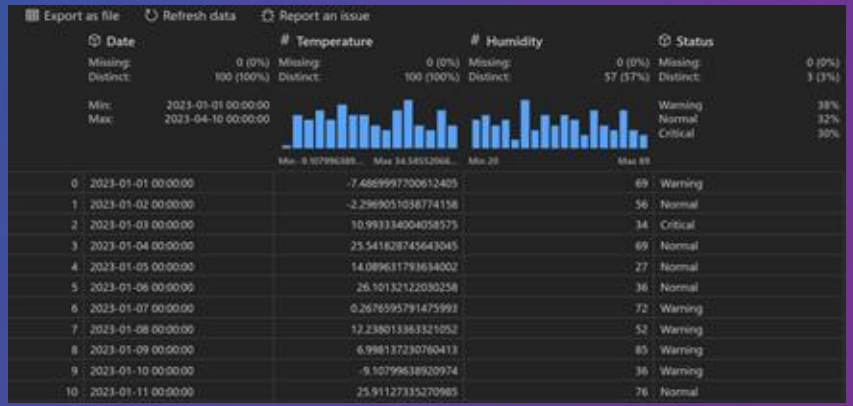
Step 2:
Check the output



Press Shift + Enter or
click the Play button

Phase 7: The Variable Explorer & Data Viewer

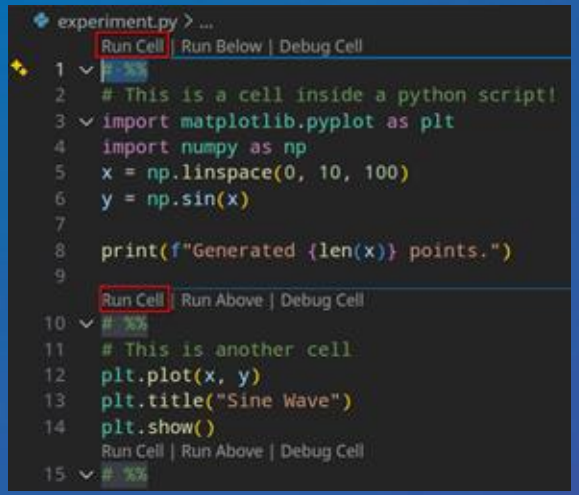


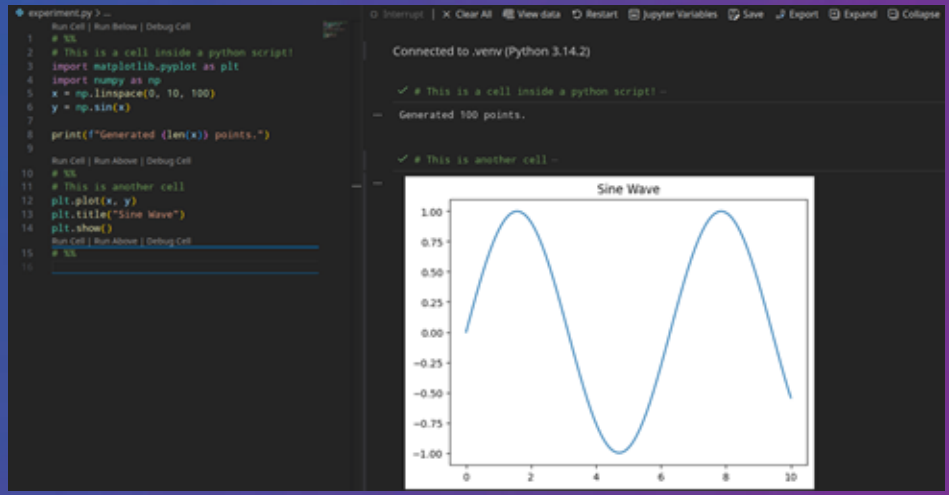


One advantage of PyCharm/Spyder was the variable viewer. VS Code now has this.
In the notebook, click the Jupyter Variables button.

The data can be sorted and filtered and can be inspected to catch errors before training a model.

Phase 8: Interactive Python Window





A standard Python script can be turned into a Jupyter Notebook by simply typing # %%

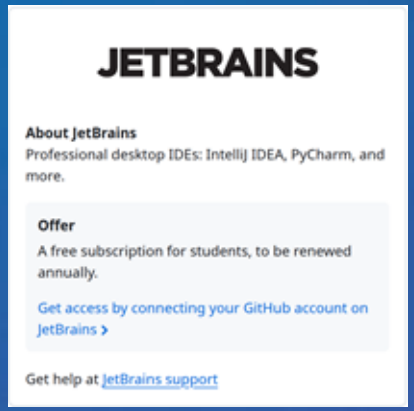
This chunk of code can be run, and its output displayed without making a mess

Get the GitHub Student Developer Pack

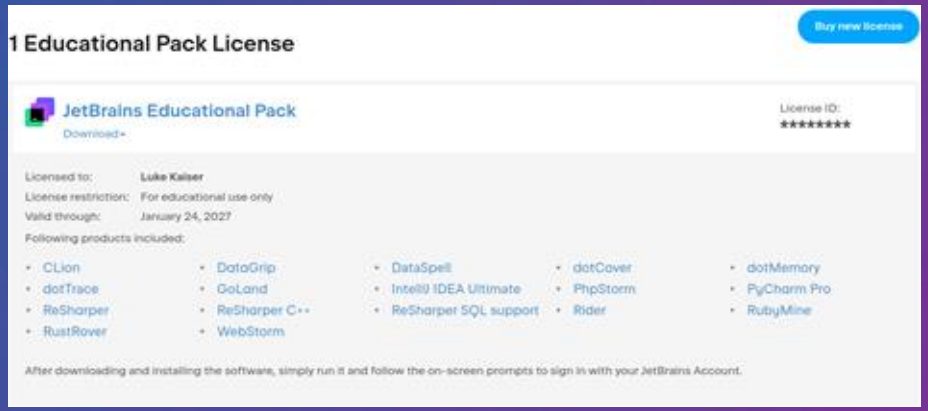
- Unlocks Copilot, PyCharm, and many other perks
- Apply: Go to the GitHub Education benefits page, click “Sign up for student developer pack,” select “Get student benefits.”
- Verification: Use your school email with your GitHub account and upload a school ID or transcript
- Wait: Approval can take a few days; you will receive an email upon approval



Activate JetBrains License



Included will be a pro license for JetBrains IDEs.



Sign in to jetbrains.com with your GitHub account to activate your Educational Pack.

Download and Install PyCharm



PyCharm can be downloaded as a standalone executable



It can also be installed and managed through the JetBrains Toolbox